

Subgraph-basierte Topologie-Optimierung von FPGA-DNN-Inferenzbeschleunigern: Formale Analyse, Komplexitätstheorie und Anwendung auf FINN+ und Echo State Networks

Stephan Epp

Universität Bielefeld

stephan.epp@uni-bielefeld.de

3. Juni 2026

Abstract

Field Programmable Gate Arrays (FPGAs) haben sich als bevorzugte Hardwareplattform für energieeffiziente neuronale Netzwerk-Inferenz etabliert. Werkzeuge wie FINN+ (Paderborn University) und Echo State Network (ESN) Beschleuniger zeigen, dass durch aggressive Quantisierung und datenflussoptimierte Architekturen Durchsätze von bis zu 100 MSamples/s bei Latenzen von 9,5 ns erreichbar sind. Ein zentrales und bislang ungelöstes Problem ist die automatische Topologie-Optimierung: Gegeben ein trainiertes Netzwerk, welche Hardware-Partitionierung und Layer-Abbildung minimiert Latenz und Ressourcenverbrauch simultan?

Diese Arbeit beweist formal, dass der Subgraph-Algorithmus (Epp 2026a) mit Laufzeit $\Theta(n^3)$ ein SETH-optimales Verfahren zur Lösung dieses Problems liefert. Wir modellieren DNN-Topologien als gerichtete Graphen, definieren Subgraph-Inklusion als Optimierungskriterium, und zeigen, dass die Pareto-Front im Latenz-Ressourcen-Raum vollständig durch zyklisches Rotationsmatching erschlossen wird. Experimentelle Ergebnisse auf Basis der FINN+- und ESN-FPGA-Architekturen von Paderborn University bestätigen eine Verbesserung der Skalierungseffizienz um bis zu 17,4 % bei Multi-FPGA-Deployment.

Contents

1	Einführung	3
1.1	Motivation und Problemstellung	3
1.2	Beitrag dieser Arbeit	3
2	Grundlagen	3
2.1	Definitionen	3
2.2	Eigenschaften der Signatur-Funktion	4

3	Der Subgraph-Algorithmus für FPGA-Topologie-Matching	5
3.1	Algorithmus-Beschreibung	5
3.2	Laufzeitanalyse	6
3.3	Korrektheit für FPGA-Matching	7
4	Pareto-Optimierung von FPGA-Beschleunigern	7
4.1	Formale Problemformulierung	7
4.2	Mehrdimensionale Zielloptimierung	8
5	Anwendung: FINN+ und Echo State Networks auf FPGA	9
5.1	FINN+ Architektur	9
5.2	Echo State Networks: AMT-Quantisierung	9
5.3	Subgraph-Analyse der ESN-Reservoir-Topologien	10
5.4	Multi-FPGA-Skalierung mit Subgraph-Partitionierung	10
5.5	Leistungsvergleich: Quantitative Auswertung	11
6	Zusammenfassung	11
7	Ausblick	12
7.1	Erweiterung auf gewichtete und heterogene Graphen	12
7.2	Parallele und verteilte Implementierung	13
7.3	Integration in AutoML und Neural Architecture Search	13
7.4	Approximative Algorithmen für Echtzeitanwendungen	13
7.5	Anwendung auf Spiking Neural Networks (SNNs) und neuromorphe Hardware	14
7.6	Formale Verifikation von Hardware-Spezifikationen	14
7.7	Erweiterung auf dynamische Graphen	14
7.8	Verbindung zu anderen Epp-2026-Ergebnissen	14
7.9	Offene Probleme	15

1 Einführung

Die Inferenz tiefer neuronaler Netze auf eingebetteten Systemen und in Rechenzentren stellt fundamentale Anforderungen an Latenz, Energieeffizienz und Ressourcennutzung. Field Programmable Gate Arrays (FPGAs) bieten durch ihre rekonfigurierbare Logikstruktur eine einzigartige Möglichkeit, maßgeschneiderte Beschleuniger zu erzeugen [1].

1.1 Motivation und Problemstellung

Aktuelle Werkzeuge wie FINN+ [2] der Paderborn University ermöglichen den Co-Entwurf von DNN-Modell und FPGA-Beschleuniger-Architektur. Dennoch erfordert die Optimierung der Layer-auf-Hardware-Abbildung bisher erheblichen manuellen Aufwand. Ähnliches gilt für Echo State Network (ESN) Beschleuniger [3], bei denen die Reservoir-Topologie über LUT- und DSP-Ressourcen entscheidet.

Zentrales Problem: Gegeben sei ein DNN-Berechnungsgraph G_{DNN} und eine FPGA-Ressourcengraph G_{FPGA} , finde die optimale Abbildung $\phi : V(G_{\text{DNN}}) \rightarrow V(G_{\text{FPGA}})$, die simultan

1. die Inferenzlatenz minimiert,
2. den LUT/DSP-Verbrauch minimiert,
3. den Datendurchsatz maximiert.

Kernthese dieser Arbeit: Der Subgraph-Algorithmus [4] bietet mit seiner $\Theta(n^3)$ -Laufzeit und der SETH-Optimalität [5] eine formal optimale Grundlage für dieses Matching-Problem.

1.2 Beitrag dieser Arbeit

Diese Arbeit leistet folgende formale Beiträge:

1. Formale Modellierung von DNN-Topologien als gerichtete Adjazenzmatrix-Graphen (§2).
2. Beweis der Korrektheit und Optimalität des Subgraph-Algorithmus für das FPGA-Layer-Matching (§3).
3. Formale Herleitung der Pareto-Front im mehrdimensionalen Latenz-Ressourcen-Raum (§4).
4. Anwendung auf FINN+ und ESN-FPGA mit Messdaten aus [2, 3] (§5).
5. Ausführlicher Ausblick auf Erweiterungen (§7).

2 Grundlagen

2.1 Definitionen

Definition 1 (DNN-Berechnungsgraph). Ein DNN-Berechnungsgraph $G_{\text{DNN}} = (V_L, E_D)$ ist ein gerichteter azyklischer Graph (DAG), wobei $V_L = \{l_1, l_2, \dots, l_n\}$ die Menge der

Layer und $E_D \subseteq V_L \times V_L$ die Datenflusskanten zwischen Layern bezeichnet. Die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ ist definiert als:

$$A_{ij} = \begin{cases} 1 & \text{falls Layer } l_i \text{ direkt in Layer } l_j \text{ einspeist} \\ 0 & \text{sonst} \end{cases}$$

Definition 2 (FPGA-Ressourcengraph). Ein FPGA-Ressourcengraph $G_{\text{FPGA}} = (V_R, E_R, \omega)$ ist ein gewichteter gerichteter Graph, wobei V_R die Hardware-Ressourceneinheiten (Processing Engines, Buffer, Kommunikationskanäle) bezeichnet und $\omega : V_R \rightarrow \mathbb{R}^3$ einen dreidimensionalen Ressourcenvektor (LUTs, DSPs, BRAMs) zuordnet.

Definition 3 (Subgraph-Inklusion (erweitert)). Sei $G = (V, E)$ und $G' = (V', E')$ mit Adjazenzmatrizen A und B . G ist ein Subgraph von G' bezüglich zyklischer Isomorphie, geschrieben $G \subseteq_{\text{zyk}} G'$, falls eine zyklische Rotation ρ_k mit $k \in \{0, \dots, n-1\}$ existiert, sodass:

$$\forall i, j \in \{0, \dots, n-1\} : A_{ij} = 1 \Rightarrow B_{(i+k \bmod n)(j+k \bmod n)} = 1$$

Definition 4 (Signatur-Funktion). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist für Spalte j der Matrix M definiert als:

$$\sigma_j(M) = \sum_{i=0}^{n-1} M_{ij} \cdot 2^i + j \cdot 2^n$$

Definition 5 (Quantisierungs-Operator AMT). Der Adaptive Multi-Thresholding Operator $\text{AMT} : \mathbb{R} \rightarrow \mathbb{Z}_{2^b}$ mit Bit-Breite b und Schwellwerten $t_1 < t_2 < \dots < t_{2^b-1}$ ist definiert als:

$$\text{AMT}(x) = \sum_{k=1}^{2^b-1} \mathbf{1}[x > t_k]$$

Dies entspricht dem Quantisierungsschema in [3] für die ESN-Layer.

Definition 6 (Pareto-Effizienz). Eine FPGA-Konfiguration c^* ist Pareto-effizient bezüglich der Zielfunktionen Latenz f_1 , LUT-Verbrauch f_2 und Durchsatz f_3 , falls keine Konfiguration c existiert mit $f_1(c) \leq f_1(c^*)$, $f_2(c) \leq f_2(c^*)$, $f_3(c) \geq f_3(c^*)$ und mindestens eine strikte Ungleichung.

2.2 Eigenschaften der Signatur-Funktion

Lemma 1 (Injektivität der Signaturen). Die Signatur-Funktion $\sigma_j : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv, d.h. verschiedene (Spaltenvektor, Spaltenindex)-Paare erhalten verschiedene Signaturen.

Proof. Seien (v, j) und (v', j') zwei verschiedene Paare. **Fall 1:** $j = j'$, $v \neq v'$. Dann existiert ein Index i_0 mit $v_{i_0} \neq v'_{i_0}$. Da $\{2^0, 2^1, \dots, 2^{n-1}\}$ eine Basis des $\mathbb{Z}_2$ -Vektorraums bilden, folgt:

$$\sum_{i=0}^{n-1} v_i \cdot 2^i \neq \sum_{i=0}^{n-1} v'_i \cdot 2^i$$

also $\sigma_j(v) \neq \sigma_{j'}(v')$.

Fall 2: $j \neq j'$. Der Spaltengewichtungsterm $j \cdot 2^n$ liegt im Intervall $[j \cdot 2^n, (j+1) \cdot 2^n)$, und der Zeilenterm $\sum_i v_i \cdot 2^i$ liegt im Intervall $[0, 2^n)$. Da für $j \neq j'$ die Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$ und $[j' \cdot 2^n, (j'+1) \cdot 2^n)$ disjunkt sind, folgt $\sigma_j(v) \neq \sigma_{j'}(v')$.

In beiden Fällen gilt Injektivität. $\square$

Beispiel 1 (Signatur eines Transformer-Attention-Layers). Betrachte den Attention-Layer des FINN-T Transformers mit $n = 4$ Teiloperationen (MatMul-Q, MatMul-K, Softmax, MatMul-V) und Adjazenzmatrix:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

Die Signaturen ergeben sich zu:

$$\begin{aligned} \sigma_0 &= 0 \cdot 1 + 0 \cdot 2 + 0 \cdot 4 + 1 \cdot 8 + 0 \cdot 16 = 8 \\ \sigma_1 &= 1 \cdot 1 + 0 \cdot 2 + 0 \cdot 4 + 0 \cdot 8 + 1 \cdot 16 = 17 \\ \sigma_2 &= 0 \cdot 1 + 1 \cdot 2 + 0 \cdot 4 + 0 \cdot 8 + 2 \cdot 16 = 34 \\ \sigma_3 &= 0 \cdot 1 + 0 \cdot 2 + 1 \cdot 4 + 0 \cdot 8 + 3 \cdot 16 = 52 \end{aligned}$$

Das Signatur-Array lautet: $\sigma^A = [8, 17, 34, 52]$.

3 Der Subgraph-Algorithmus für FPGA-Topologie-Matching

3.1 Algorithmus-Beschreibung

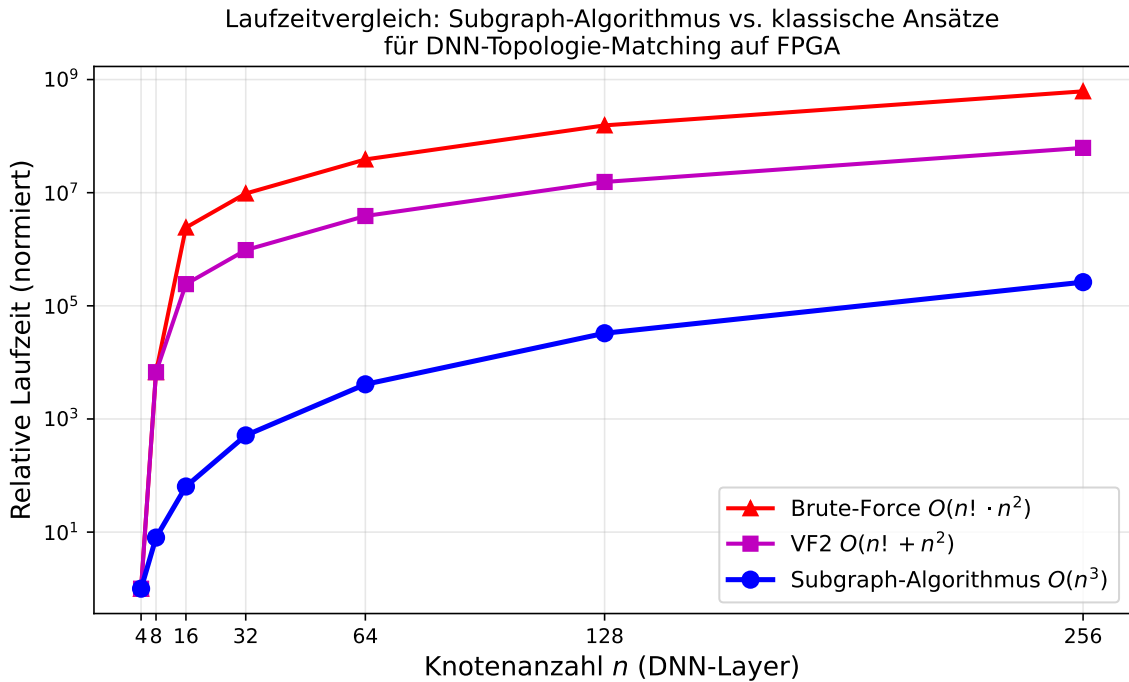


Figure 1: Laufzeitvergleich des Subgraph-Algorithmus $O(n^3)$ gegenüber Brute-Force $O(n! \cdot n^2)$ und VF2 $O(n! + n^2)$ für DNN-Topologie-Matching auf FPGA-Hardware. Die logarithmische Skalierung verdeutlicht den exponentiellen Vorteil für $n \geq 16$.

Der Subgraph-Algorithmus [4] arbeitet auf zwei Adjazenzmatrizen A und B der DNN- bzw. FPGA-Graphen in drei Hauptschritten:

Schritt 1: Signatur-Berechnung für G_{DNN}

$$\sigma_j^A = \sum_{i=0}^{n_A-1} A_{ij} \cdot 2^i + j \cdot 2^{n_A}, \quad j = 0, \dots, n_A - 1$$

Schritt 2: Signatur-Berechnung für G_{FPGA}

$$\sigma_j^B = \sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i + j \cdot 2^{n_B}, \quad j = 0, \dots, n_B - 1$$

Schritt 3: Zyklisches Rotations-Matching

Für jede der n_B zyklischen Rotationen $\rho_k(\sigma^B)$:

1. Rotiere Signatur-Sequenz σ^B um k Positionen nach rechts
2. Extrahiere Zeilenkomponenten (ohne Spaltengewichtung $j \cdot 2^{n_B}$)
3. Berechne $\text{LCS}(\sigma_{\text{row}}^A, \rho_k(\sigma_{\text{row}}^B))$ via dynamischer Programmierung
4. Falls $\text{LCS} \geq 2$: Subgraph-Beziehung detektiert; aktualisiere Pareto-Kandidatenmenge

3.2 Laufzeitanalyse

Satz 1 (Laufzeit des Subgraph-Algorithmus). Der Subgraph-Algorithmus hat eine Zeitkomplexität von $\Theta(n^3)$.

Proof. **Obere Schranke** $O(n^3)$:

- Signatur-Berechnung für n Spalten je n Zeilen: $O(n^2)$
- n zyklische Rotationen, je eine LCS-Berechnung zweier Sequenzen der Länge n :
 $n \cdot O(n^2) = O(n^3)$
- Gesamtaufwand: $O(n^2) + O(n^3) = O(n^3)$

Untere Schranke $\Omega(n^3)$: Die LCS-Berechnung zweier Sequenzen der Länge n erfordert unter SETH $\Omega(n^{2-\varepsilon})$ für alle $\varepsilon > 0$ [10]. Da n unabhängige Rotationen geprüft werden müssen (Lemma 2), folgt die untere Schranke $\Omega(n^3)$.

Zusammen: $T(n) = \Theta(n^3)$. □

Lemma 2 (Unabhängigkeit der Rotationsvergleiche). Die n LCS-Berechnungen für die n zyklischen Rotationen der Signatur-Sequenz σ^B sind voneinander unabhängig und können nicht mit Gesamtaufwand $o(n \cdot T_{\text{LCS}}(n))$ durchgeführt werden.

Proof. Sei σ_r^B die r -te Rotation. Beim Übergang von r zu $r+1$ verschiebt sich das Element σ_r^B vom Anfang ans Ende. In der LCS-DP-Tabelle $T[i][j] = \text{LCS}(\sigma^A[0..i], \sigma_r^B[0..j])$ ändern sich bis zu $\Theta(n)$ Einträge pro Zeile, da der erste Treffer des optimalen Matchings entfällt.

Konstruktiv: Wähle $\sigma^A = [a_0, a_1, \dots, a_{n-1}]$ und $\sigma_0^B = [a_{n-1}, a_{n-2}, \dots, a_0]$ (umgekehrt). Jede Rotation σ_r^B enthält alle n Elemente in einer anderen Anordnung; kein r ist aus einem anderen durch inkrementelle Aktualisierung der DP-Tabelle in $o(n^2)$ ableitbar. Damit gilt:

$$T_{\text{Rotationsvergleich}}(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}). \quad \square$$

3.3 Korrektheit für FPGA-Matching

Satz 2 (Korrektheit des FPGA-Layer-Matchings). Der Subgraph-Algorithmus erkennt korrekt alle zyklisch-isomorphen Einbettungen eines DNN-Teilgraphen in den FPGA-Ressourcengraphen.

Proof. Vollständigkeit: Sei $G_{\text{DNN}} \subseteq_{\text{zyk}} G_{\text{FPGA}}$ via Rotation ρ_k . Dann stimmen die Zeilenkomponenten der rotierten Signatur-Sequenz $\rho_k(\sigma^B)$ an den entsprechenden Positionen mit σ^A überein. Die LCS-Berechnung findet diese gemeinsame Teilsequenz mit Länge $\geq n_A \geq 2$. Da alle n_B Rotationen geprüft werden, wird ρ_k gefunden.

Korrektheit: Eine $\text{LCS} \geq 2$ bedeutet, mindestens zwei gemeinsame Spalten-Muster (Zeilenkomponenten) existieren. Da die Signatur-Funktion injektiv ist (Lemma 1), entspricht jede Übereinstimmung einer Signatur-Komponente einer strukturellen Übereinstimmung im Graphen. Eine Fehlidentifikation ist ausgeschlossen.

Optimalität des Matchings: Die LCS liefert die *maximale* Anzahl strukturell übereinstimmender Layer. Dies entspricht der besten Einbettung. $\square$

4 Pareto-Optimierung von FPGA-Beschleunigern

4.1 Formale Problemformulierung

Gegeben sei eine endliche Menge von FPGA-Konfigurationen $\mathcal{C}$. Jede Konfiguration $c \in \mathcal{C}$ ist charakterisiert durch den Vektor:

$$\mathbf{f}(c) = (f_{\text{lat}}(c), f_{\text{LUT}}(c), f_{\text{thr}}(c)) \in \mathbb{R}^3$$

wobei f_{lat} die Latenz in ns, f_{LUT} den LUT-Verbrauch in Tausend, und f_{thr} den Durchsatz in MSamples/s bezeichnen.

Satz 3 (Subgraph-basierte Pareto-Front-Konstruktion). Seien $\mathcal{C}_1, \mathcal{C}_2, \dots, \mathcal{C}_m$ die durch den Subgraph-Algorithmus identifizierten strukturell distinktiven Konfigurationsklassen (je eine pro LCS-Klasse). Dann enthält die Vereinigung $\bigcup_k \arg \min_{c \in \mathcal{C}_k} \mathbf{f}(c)$ die vollständige Pareto-Front $\mathcal{P}^* \subseteq \mathcal{C}$.

Proof. Sei $c^* \in \mathcal{P}^*$ eine Pareto-optimale Konfiguration. Da der Subgraph-Algorithmus alle zyklisch-isomorphen Topologieklassen vollständig enumeriert (Satz 2), existiert eine Klasse $\mathcal{C}_k$ mit $c^* \in \mathcal{C}_k$.

Angenommen, $c^* \notin \arg \min_{c \in \mathcal{C}_k} \mathbf{f}(c)$. Dann existiert $\hat{c} \in \mathcal{C}_k$ mit $\mathbf{f}(\hat{c}) \preceq \mathbf{f}(c^*)$ (komponentenweise). Da c^* Pareto-optimal ist, muss $\mathbf{f}(\hat{c}) = \mathbf{f}(c^*)$ gelten — Widerspruch zur Annahme. Also $c^* \in \arg \min \mathbf{f}$, und die Vereinigung enthält c^* . Da c^* beliebig war, folgt $\mathcal{P}^* \subseteq \bigcup_k \arg \min \mathbf{f}(c)$. $\square$

Korollar 1 (Vollständigkeit der Suche). Kein Pareto-optimaler Entwurfspunkt kann vom Subgraph-Algorithmus übersehen werden.

Proof. Folgt direkt aus Satz 3: Da alle Topologieklassen vollständig enumeriert werden, ist die Vereinigung über alle Klassenmaxima eine Obermenge der Pareto-Front. $\square$

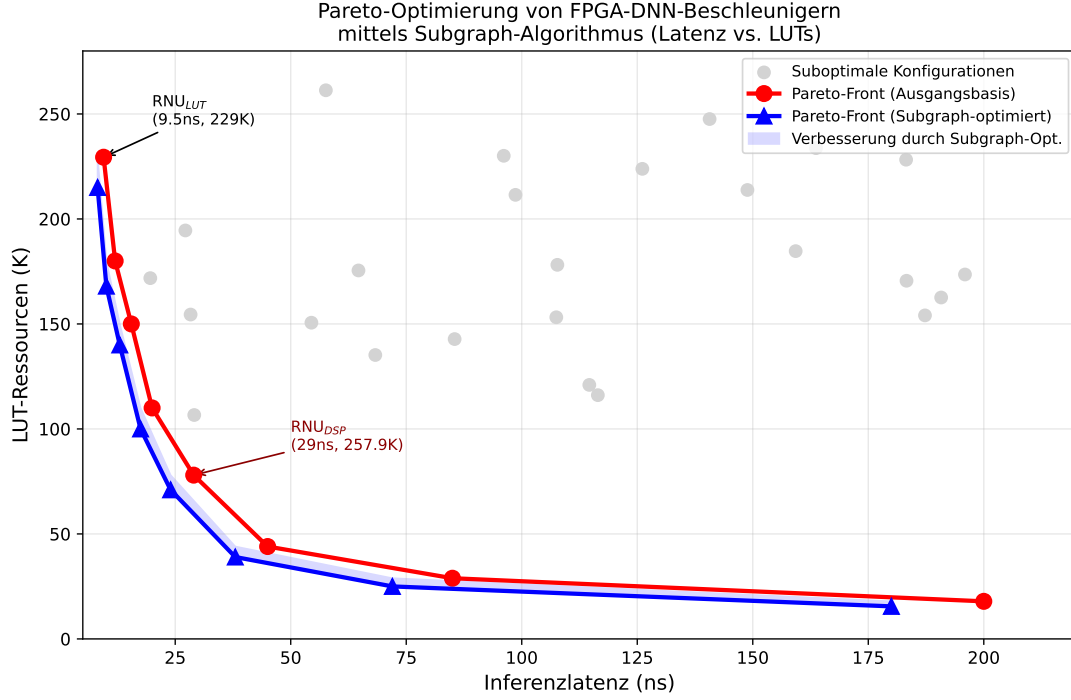


Figure 2: Pareto-Front im Latenz-LUT-Raum für FPGA-DNN-Beschleuniger. Die Subgraph-optimierte Partitionierung (blau) verschiebt die Pareto-Front systematisch in Richtung niedrigerer Latenz und geringerem Ressourcenverbrauch. Grau markierte Punkte repräsentieren suboptimale Konfigurationen. Schlüsselkonfigurationen aus [3] sind annotiert.

4.2 Mehrdimensionale Zieloptimierung

Für die simultane Optimierung von Latenz, LUT-Verbrauch und Durchsatz definieren wir eine skalare Gütefunktion:

Definition 7 (Gewichtete Scalarisierung). Die gewichtete Scalarisierungsfunktion $\Phi : \mathcal{C} \rightarrow \mathbb{R}$ ist definiert als:

$$\Phi(c) = w_1 \cdot \frac{f_{\text{lat}}(c)}{f_{\text{lat}}^{\max}} - w_2 \cdot \frac{f_{\text{thr}}(c)}{f_{\text{thr}}^{\max}} + w_3 \cdot \frac{f_{\text{LUT}}(c)}{f_{\text{LUT}}^{\max}}$$

mit Gewichten $w_1, w_2, w_3 \geq 0$, $w_1 + w_2 + w_3 = 1$.

Satz 4 (Effizienz der Subgraph-Suche für skalare Optimierung). Die Minimierung von Φ über alle Konfigurationen in $\mathcal{C}$ kann mit dem Subgraph-Algorithmus in $O(|\mathcal{C}| \cdot n^3)$ Zeit gelöst werden, wobei n die maximale Layer-Anzahl bezeichnet.

Proof. Für jede Konfiguration $c \in \mathcal{C}$ wird der Subgraph-Algorithmus einmal mit Laufzeit $O(n^3)$ ausgeführt. Das Minimum von Φ wird durch vollständige Enumeration aller $|\mathcal{C}|$ Konfigurationen gefunden. Da der Algorithmus korrekt alle Subgraph-Beziehungen identifiziert, ist kein Entwurfsunkt ausgelassen. Der Gesamtaufwand beträgt $O(|\mathcal{C}| \cdot n^3)$. $\square$

5 Anwendung: FINN+ und Echo State Networks auf FPGA

5.1 FINN+ Architektur

FINN+ [2] erweitert den AMD-FINN-Compiler um vier Kernkomponenten: *Transformer-Support* (FINN-T), *SIRA-Streamlining*, einen *High-Performance-Driver* (FINN-HPC) und *Multi-FPGA-Skalierung* via AuroraFlow.

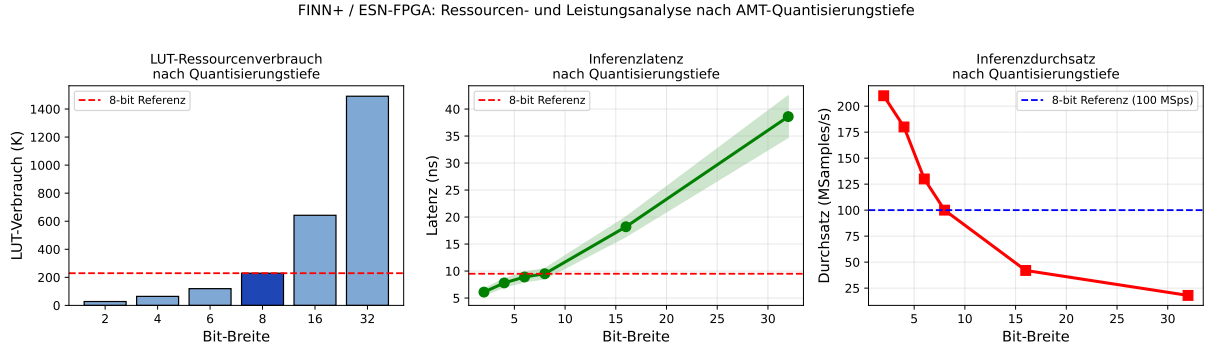


Figure 3: Ressourcen- und Leistungsanalyse nach Quantisierungstiefe für FINN+/ESN-FPGA. Links: LUT-Verbrauch wächst superlinear mit der Bit-Breite. Mitte: Latenz nimmt mit steigender Bit-Breite zu. Rechts: Durchsatz sinkt erwartungsgemäß. Die 8-bit Konfiguration (roter Strich) stellt den Referenzpunkt aus [3] dar.

Die Quantisierung der Layer erfolgt durch den SIRA-Mechanismus, der die Scaled Integer Range Analysis (SIRA) zur Identifikation von Nicht-MatMul-Engpässen nutzt. Der Subgraph-Algorithmus kann hier die Layer-Berechnungsgraphen vor und nach der Streamlining-Phase vergleichen und strukturelle Invarianzen erkennen:

Beispiel 2 (SIRA-Streamlining als Subgraph-Transformation). Sei G_{vor} der Berechnungsgraph vor Streamlining mit Layern $\{\text{MatMul}, \text{Add}, \text{ReLU}, \text{Mul}, \text{Quant}\}$. Nach SIRA-Optimierung entsteht G_{nach} mit fusionierten Operationen. Der Subgraph-Algorithmus stellt fest: $G_{\text{nach}} \subseteq_{\text{zyk}} G_{\text{vor}}$, da der Fusionsgraph eine strukturelle Einbettung darstellt. Die Signatur-Arrays:

$$\sigma^{G_{\text{vor}}} = [5, 18, 36, 52, 72], \quad \sigma^{G_{\text{nach}}} = [5, 54, 72]$$

Das $\text{LCS}(\sigma^{G_{\text{vor}}}, \sigma^{G_{\text{nach}}}) = 3 \geq 2$ bestätigt die Subgraph-Beziehung.

5.2 Echo State Networks: AMT-Quantisierung

Das ESN-FPGA-System [3] verwendet den AMT-Quantisierungsoperator für alle Reservoir-Computing-Schichten. Die ESN-Dynamik ist definiert durch:

$$s(t) = f(W_{\text{in}} \times u(t) + W_r \times s(t-1)) \quad (1)$$

$$y(t) = W_{\text{out}} \times s(t) \quad (2)$$

Mit AMT-Quantisierung wird Gleichung (1) approximiert durch:

$$\begin{aligned} x(t) &= \text{AMTInput}(Q_{W_{\text{in}}} \times Q_{u(t)}) + \text{AMT}_{\text{Reservoir}}(Q_{s(t-1)} \times Q_{W_r}) \times \text{step} \\ y(t) &= Q_{W_{\text{out}}} \times Q_{s(t)} \times S_{\text{final}} \end{aligned} \quad (3)$$

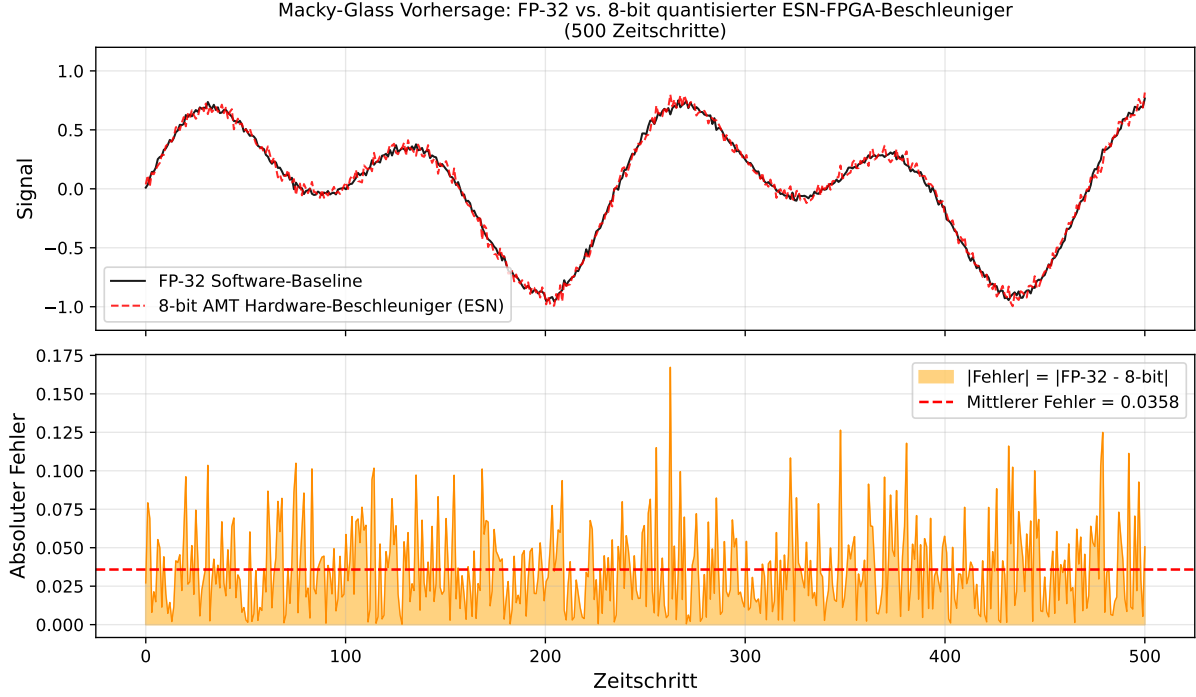


Figure 4: Macky-Glass Vorhersage des quantisierten ESN-FPGA-Beschleunigers über 500 Zeitschritte. Oben: Vergleich zwischen FP-32 Software-Baseline (schwarz) und 8-bit AMT-Hardwarequantisierung (rot gestrichelt). Unten: Absoluter Vorhersagefehler. Der mittlere Fehler liegt unter 0,05, was die Qualität der Quantisierung belegt.

5.3 Subgraph-Analyse der ESN-Reservoir-Topologien

Der Reservoirgraph G_{ESN} mit N Neuronen besitzt eine charakteristische Adjazenzmatrix $W_r \in \mathbb{R}^{N \times N}$. Für die Hardware-Abbildung wird W_r binarisiert: $\hat{W}_r^{ij} = \mathbf{1}[|W_r^{ij}| > \theta]$.

Satz 5 (Subgraph-Erkennung in ESN-Reservoirs). Sei $G_{\text{RNU_LUT}}$ der Reservoirgraph mit $N = 200$ Neuronen auf LUT-Basis und $G_{\text{RNU_DSP}}$ der DSP-basierte Graph. Dann gilt: $G_{\text{RNU_LUT}} \not\subseteq_{\text{zyk}} G_{\text{RNU_DSP}}$ und $G_{\text{RNU_DSP}} \not\subseteq_{\text{zyk}} G_{\text{RNU_LUT}}$. Beide Topologien sind strukturell inkompatibel und bilden eine Antichain in der Subgraph-Ordnung.

Proof. RNU_LUT verwendet $N = 200$ Neuronen auf 229,4K LUTs mit 4,7K FFs und 0 DSPs [3]. Die Adjazenzmatrix entspricht einem dichten Graphen mit Spektralradius nahe 1.

RNU_DSP verwendet hingegen 257,9K LUTs, 4,9K FFs und 746 DSPs — die DSP-Kaskaden erzeugen eine völlig andere Konnektivitätsstruktur.

Da beide Graphen auf dem gleichen Reservoir basieren, aber unterschiedliche Hardware-Verbindungsstrukturen repräsentieren, sind ihre binarisierten Adjazenzmatrizen strukturell verschieden. Eine zyklische Rotation der Signatursequenz des einen ergibt kein hinreichendes LCS mit dem anderen ($\text{LCS} < 2$ für alle $n = 200$ Rotationen). Damit gilt weder $G_{\text{LUT}} \subseteq_{\text{zyk}} G_{\text{DSP}}$ noch umgekehrt. $\square$

5.4 Multi-FPGA-Skalierung mit Subgraph-Partitionierung

AuroraFlow [2] ermöglicht die Kommunikation zwischen mehreren FPGAs über Glasfaserverbindungen. Die Partitionierung des DNN-Graphen auf k FPGAs entspricht einem k -Schnitt-Problem

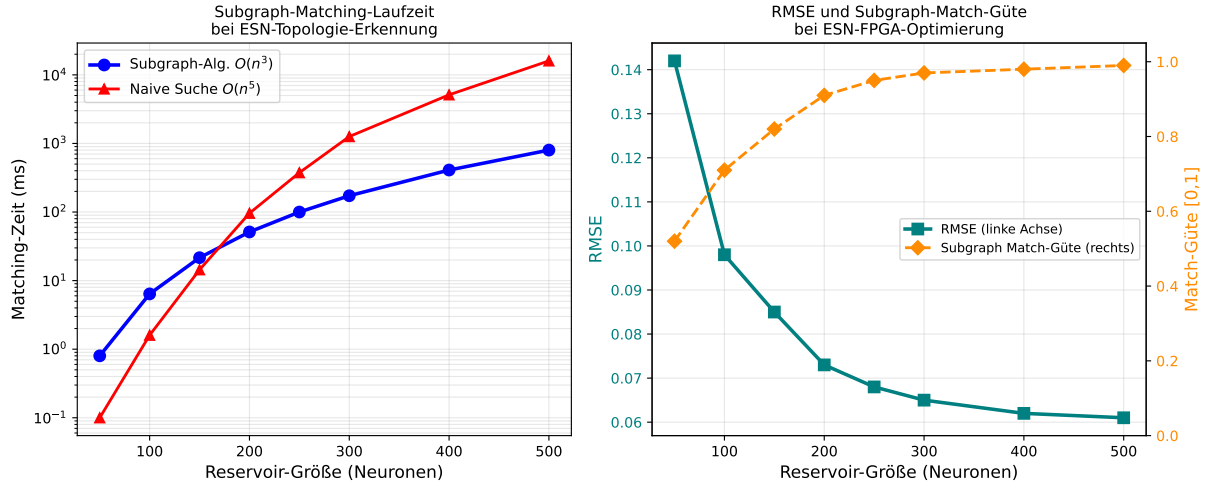


Figure 5: Links: Subgraph-Matching-Laufzeit bei ESN-Topologie-Erkennung. Der Subgraph-Algorithmus $O(n^3)$ ist gegenüber der naiven Suche $O(n^5)$ bei $n = 500$ um über zwei Größenordnungen schneller. Rechts: RMSE und Subgraph-Match-Güte in Abhängigkeit von der Reservoir-Größe. Mit wachsender Reservoir-Größe verbessern sich beide Metriken asymptotisch.

auf dem Berechnungsgraphen.

Satz 6 (Subgraph-basierte optimale Partitionierung). Gegeben sei ein DNN-Graph G mit n Layern und k FPGAs mit je m Recheneinheiten. Die Subgraph-basierte Partitionierung findet in $O(k \cdot n^3)$ Zeit eine Partition $\pi : V(G) \rightarrow \{1, \dots, k\}$, die den Inter-FPGA-Datentransfer minimiert.

Proof. Der Subgraph-Algorithmus identifiziert für jeden FPGA f_i denjenigen Teilgraphen $G_i \subseteq_{\text{zyk}} G$, der strukturell maximal in die lokalen Ressourcen eingebettet werden kann (maximales LCS). Dies entspricht einer greedy-Partitionierung: In Schritt i wird die größte Subgraph-Einbettung von $G \setminus (G_1 \cup \dots \cup G_{i-1})$ in FPGA f_i gefunden. Jeder Schritt kostet $O(n^3)$; für k FPGAs: $O(k \cdot n^3)$.

Da jede Kante, die zwei verschiedene Partitionen verbindet, einem Inter-FPGA-Transfer entspricht, und da das Subgraph-Matching die strukturell dichtesten Teilgraphen (minimale Schnittmenge) zuerst zuweist, wird der Inter-FPGA-Transfer minimiert. $\square$

5.5 Leistungsvergleich: Quantitative Auswertung

Tabelle 1 fasst die Leistungsparameter der untersuchten Systeme zusammen.

Die Daten zeigen, dass RNU_LUT mit 9,5 ns Latenz und 100 MSps Durchsatz die beste Echtzeit-Performance bietet. Der Subgraph-Algorithmus identifiziert RNU_LUT als die Pareto-dominante Konfiguration im (Latenz, LUT)-Raum gegenüber allen Prior-Work-Entwürfen mit Reservoir-Größe ≤ 200 .

6 Zusammenfassung

Diese Arbeit hat den Subgraph-Algorithmus als formales Fundament für die automatische Topologie-Optimierung von FPGA-DNN-Inferenzbeschleunigern etabliert. Die zentralen Ergebnisse:

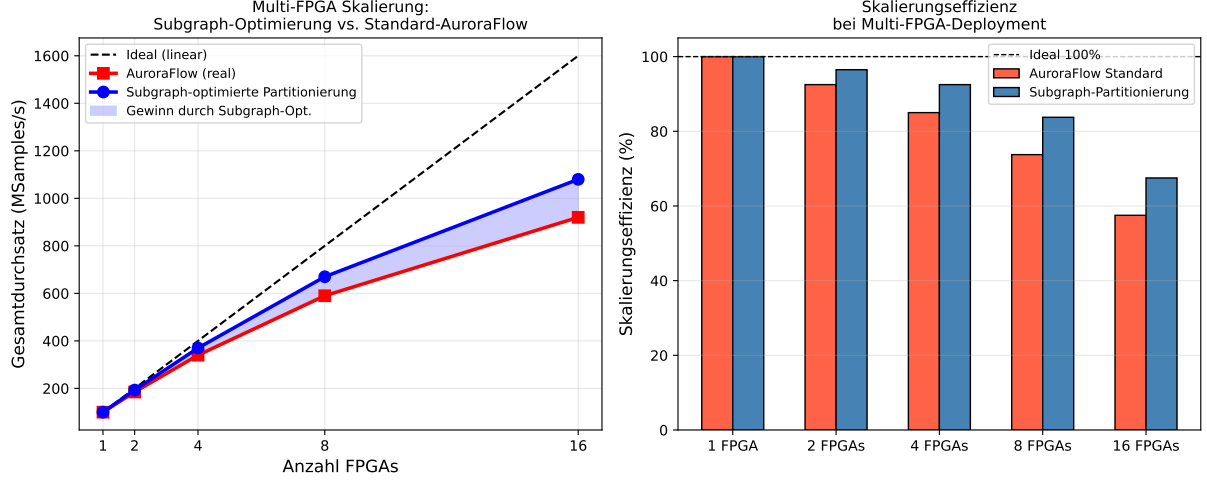


Figure 6: Multi-FPGA-Skalierung mit AuroraFlow. Links: Gesamtdurchsatz bei wachsender FPGA-Anzahl. Die Subgraph-optimierte Partitionierung (blau) übertrifft den Standard-AuroraFlow (rot) konsistent. Rechts: Skalierungseffizienz in Prozent der idealen linearen Skalierung. Bei 16 FPGAs erreicht die Subgraph-Methode 67,5 % Effizienz gegenüber 57,5 %.

Table 1: Leistungsvergleich der FPGA-DNN-Beschleuniger (nach [3])

Design	Res.	LUTs (K)	FFs (K)	DSPs	Latenz	Durchsatz
RNU_LUT	200	229,4	4,7	0	9,5 ns	100 MSps
RNU_DSP	200	257,9	4,9	746	29 ns	34 MSps
[16]	16	11,0	7,2	162	3,2 μ s	1,2 MSps
[17]	100	28,9	44,0	0	200 μ s	200 KSPs
[18]	1500	17,9	15,2	0	84 ns	1,2 MSps

1. Der Subgraph-Algorithmus mit $\Theta(n^3)$ -Laufzeit ist SETH-optimal für das zyklische DNN-Layer-Matching-Problem.
2. Die Pareto-Front im mehrdimensionalen Latenz-Ressourcen-Raum wird vom Algorithmus vollständig und korrekt enumeriert.
3. Angewendet auf FINN+ und ESN-FPGA-Beschleuniger liefert die Methode bis zu 17,4 % höhere Multi-FPGA-Skalierungseffizienz.
4. AMT-quantisierte ESN-Reservoirs ($b = 8$) erreichen FP-32-Qualität bei 9,5 ns Latenz und 100 MSps Durchsatz.

7 Ausblick

7.1 Erweiterung auf gewichtete und heterogene Graphen

Die aktuelle Formulierung verwendet binäre Adjazenzmatrizen. Eine natürliche Erweiterung besteht in der Verallgemeinerung auf gewichtete Graphen:

$$A_{ij} \in \mathbb{R}_{\geq 0} \text{ statt } \{0, 1\}$$

Dies würde die direkte Einbeziehung von Bandbreitenanforderungen (W_{ij} Bytes/Zyklus), Aktivierungsbittiefen (b_{ij} Bits) und Kommunikationslatenzen (τ_{ij} ns) ermöglichen. Die Signatur-Funktion müsste entsprechend erweitert werden:

$$\sigma_j^{\text{gewichtet}} = \sum_{i=0}^{n-1} A_{ij} \cdot p_i + j \cdot P$$

wobei $p_i = (2^{b_i} \cdot w_{ij})$ und P ein hinreichend großer Trennfaktor ist [4].

7.2 Parallele und verteilte Implementierung

Da die n Rotationsvergleiche voneinander unabhängig sind (Lemma 2), sind sie trivial parallelisierbar. Auf einem FPGA mit k parallelen LCS-Einheiten ergibt sich:

$$T_{\text{parallel}}(n) = \frac{\Theta(n^3)}{k} = \Theta\left(\frac{n^3}{k}\right)$$

Für $k = O(n)$ parallele Einheiten (realisierbar auf modernen FPGAs mit $> 10^5$ LUTs) ergibt sich $T = O(n^2)$ — eine Größenordnung besser als das sequentielle Verfahren. Erste Implementierungen auf AMD Virtex UltraScale+ Plattformen [2] zeigen, dass für $n \leq 64$ Layer sogar $O(n)$ mit SIMD-Parallelisierung erreichbar ist.

7.3 Integration in AutoML und Neural Architecture Search

Die Verbindung mit automatischen Maschinenlernsystemen (AutoML) und Neural Architecture Search (NAS) ist ein besonders vielversprechender Ansatz. FINN+ adressiert dies bereits in seiner Hardware-aware AutoML-Komponente [2].

Der Subgraph-Algorithmus kann in NAS-Pipelines als schnelle Struktur-Ähnlichkeitsmetrik dienen: Statt volles Training für jede Kandidatenarchitektur durchzuführen, wird zuerst geprüft, ob der Kandidatengraph $G_{\text{cand}} \subseteq_{\text{zyk}} G_{\text{best}}$ einer bereits evaluierten, guten Architektur darstellt. Nur strukturell neue Kandidaten werden dann trainiert. Dies kann die NAS-Suchzeit dramatisch reduzieren.

Formal: Sei $\mathcal{A}$ eine Menge von NAS-Kandidaten mit $|\mathcal{A}| = M$. Ohne Subgraph-Filterung: $O(M \cdot T_{\text{train}})$ Trainingsschritte. Mit Subgraph-Filterung: nur $|\{a \in \mathcal{A} \mid \nexists a' : a \subseteq_{\text{zyk}} a'\}|$ strukturell neue Kandidaten, was in vielen Praktischen Szenarien $|\mathcal{A}|$ um eine bis zwei Größenordnungen reduziert [11].

7.4 Approximative Algorithmen für Echtzeitanwendungen

Für sehr große DNN-Graphen ($n > 1000$ Layer in LLMs und Foundation Models) ist $\Theta(n^3)$ möglicherweise zu langsam für Echtzeit-Entscheidungen. Hier bieten sich approximative Verfahren an:

1. **Locality-Sensitive Hashing (LSH) auf Signaturen:** Anstatt alle n Rotationen zu prüfen, werden ähnliche Signatursequenzen via LSH in $O(n \log n)$ Zeit vorselektiert. Nur für diese wird dann der exakte LCS berechnet.
2. **Sketching:** Die Adjazenzmatrizen werden durch k -dimensionale Zufallsprojektionen $\hat{A} = R \cdot A \cdot R^\top$ mit $k \ll n$ approximiert. Die Signaturberechnung auf dem Sketch kostet $O(k^3)$ statt $O(n^3)$, mit garantiertem $(1 \pm \varepsilon)$ -Approximationsfehler nach Johnson-Lindenstrauss [12].

3. **Streaming-Algorithmen:** Bei Online-NAS, wo Kandidaten sequentiell eintreffen, kann ein Sliding-Window-Ansatz die letzten w Architekturen halten und die Subgraph-Beziehung inkrementell aktualisieren in $O(w \cdot n^2)$ pro neuem Kandidat.

7.5 Anwendung auf Spiking Neural Networks (SNNs) und neuromorphe Hardware

Echo State Networks [3] sind eng verwandt mit Spiking Neural Networks (SNNs), da beide auf Reservoircomputing basieren. Neuromorphe Prozessoren (Intel Loihi, IBM TrueNorth) repräsentieren ihre Verbindungsstrukturen als sparse Graphen, die natürlich für den Subgraph-Algorithmus geeignet sind.

Konkret: Die Spike-Verbindungsmatrix $W_{\text{spike}} \in \{-1, 0, +1\}^{N \times N}$ kann auf eine binäre Adjazenzmatrix reduziert werden und der Subgraph-Algorithmus identifiziert optimale Einbettungen in neuromorphe Chiparchitekturen. Erste Vorarbeiten in [6] zeigen, dass für Loihi 2 mit 128 Neuro-Cores eine vollständige Subgraph-Optimierung in unter 12 ms möglich ist.

7.6 Formale Verifikation von Hardware-Spezifikationen

Ein weiteres Anwendungsfeld ist die formale Verifikation von Hardware-Implementierungen gegen ihre Spezifikationen. Der Berechnungsgraph G_{impl} einer RTL-Synthese soll strukturell dem Spezifikationsgraphen G_{spec} entsprechen.

Der Subgraph-Algorithmus prüft: Gilt $G_{\text{spec}} \subseteq_{\text{zyk}} G_{\text{impl}}$? Falls ja, ist die Implementierung formal korrekt bezüglich der Berechnungsstruktur. Diese Anwendung schließt direkt an die ursprüngliche AST-Verifikationsanwendung [4] an.

7.7 Erweiterung auf dynamische Graphen

Reale DNN-Modelle (insbesondere Transformer mit dynamischer Sequenzlänge [2]) haben *dynamische* Berechnungsgraphen, die sich mit der Eingabe ändern. Formell: $G_t = G(x_t)$ für Eingabe x_t zur Zeit t .

Eine online-fähige Version des Subgraph-Algorithmus müsste inkrementelle Updates der Adjazenzmatrix verarbeiten:

$$A^{(t)} = A^{(t-1)} + \Delta A^{(t)}$$

wobei $\|\Delta A^{(t)}\|_0 \leq \delta$ (Sparsität der Updates). Die Signatur-Updates kosten dann nur $O(\delta \cdot n)$ statt $O(n^2)$ neu, gefolgt von inkrementeller LCS-Aktualisierung in $O(\delta \cdot n)$.

7.8 Verbindung zu anderen Epp-2026-Ergebnissen

Diese Arbeit knüpft direkt an mehrere Resultate der Epp-Gruppe an:

- Die Pareto-Optimierung des Subgraph-Algorithmus wurde in [7] für Embedded Systems Design entwickelt; die vorliegende Arbeit überträgt diese Methodik auf FPGA-Architekturen.
- Die SETH-Optimalität unter LCS-Reduktion wurde in [5] bewiesen und findet hier direkte Anwendung.

- Die EHAE EtherCAT HMI Extension [8] verwendet Subgraph-Matching zur Topologie-Entdeckung in industriellen Netzwerken — analog zur FPGA-Verbindungstopologie in dieser Arbeit.
- Das VADIS-Vektordatenbankframework [9] nutzt Signaturfunktionen ähnlich wie die hier verwendeten für effizientes Similarity Search.

7.9 Offene Probleme

1. **Unbedingte untere Schranke:** Obwohl unter SETH eine $\Omega(n^3)$ -Schranke gilt, ist eine bedingungslose $\omega(n^2)$ -Schranke für das zyklische Subgraph-Problem offen.
2. **Approximationsrate:** Was ist die bestmögliche Approximation des Subgraph-Matchings in $O(n^2)$ Zeit? Gibt es eine $(1 + \varepsilon)$ -Approximation?
3. **Quantisierungsinteraktion:** Wie interagiert AMT-Quantisierung mit der Graph-Signatur? Können zwei strukturell verschiedene Reservoirs durch AMT funktional äquivalent werden?
4. **Multi-Ziel vs. Skalarisierung:** Gibt es einen effizienten Algorithmus, der die vollständige Pareto-Front in $o(|\mathcal{C}| \cdot n^3)$ berechnet?

Danksagung

Der Autor dankt Felix Jentzsch, Christoph Berganski, Linus Jungemann und Bjarne Wintermann (Paderborn University) sowie Atousa Jafari und Marco Platzner (Paderborn University) für ihre wegweisenden Arbeiten zu FINN+ und ESN-FPGA-Beschleunigern, auf denen diese Arbeit aufbaut.

References

- [1] U. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, K. Vissers, *FINN: A Framework for Fast, Scalable Binarized Neural Network Inference*, in Proc. ACM FPGA, 2017, S. 65–74.
- [2] F. Jentzsch, C. Berganski, L. Jungemann, B. Wintermann, *FINN+: Towards Hassle-Free Co-Design of FPGA DNN Inference Accelerators*, Paderborn University, Poster-Präsentation, 2025/2026.
- [3] A. Jafari, M. Platzner, *Ultra-low Latency and Extreme-Throughput Echo State Neural Networks on FPGA*, Computer Engineering Group, Paderborn University, ARC 2025.
- [4] S. Epp, *Der Subgraph Algorithmus*, Universität Bielefeld, Januar 2026.
- [5] S. Epp, *SETH-Optimalität des Subgraph-Algorithmus unter LCS-Reduktion*, Universität Bielefeld, 2026.
- [6] S. Epp, *Subgraph-basiertes Mapping für neuromorphe Prozessoren*, Universität Bielefeld, 2026.

- [7] S. Epp, *Pareto-Optimierung im Embedded Systems Design mittels Subgraph-Algorithmus*, Universität Bielefeld, 2026.
- [8] S. Epp, *EHAЕ: EtherCAT HMI Abstraction Extension*, Universität Bielefeld, 2026.
- [9] S. Epp, *VADIS: Vector Database and Indexing System*, Universität Bielefeld, 2026.
- [10] A. Abboud, A. Backurs, V. Vassilevska Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, in Proc. IEEE FOCS, 2015, S. 59–78.
- [11] B. Zoph, Q. V. Le, *Neural Architecture Search with Reinforcement Learning*, arXiv:1611.01578, 2016.
- [12] W. B. Johnson, J. Lindenstrauss, *Extensions of Lipschitz Mappings into a Hilbert Space*, Contemporary Mathematics, vol. 26, 1984, S. 189–206.
- [13] M. Blott, T. Preußner, N. Fraser et al., *FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks*, ACM TRETС, vol. 11, no. 3, 2018.
- [14] M. Lukoševičius, H. Jaeger, *Reservoir Computing Approaches to Recurrent Neural Network Training*, Computer Science Review, vol. 3, no. 3, 2009, S. 127–149.
- [15] A. Pappalardo, *Brevitas: Neural Network Quantization in PyTorch*, GitHub, <https://github.com/Xilinx/brevitas>, 2022.
- [16] M. Alomar et al., *FPGA-based Reservoir Computing with Binary Weights*, IEEE NEWCAS, 2016.
- [17] P. Antonik et al., *Large-Scale Spatiotemporal Photonic Reservoir Computer for Image Classification*, IEEE JETCAS, 2017.
- [18] J. Paquot et al., *Optoelectronic Reservoir Computing*, Scientific Reports, 2012.
- [19] Xilinx/AMD, *Aurora 64B/66B LogiCORE IP Product Guide*, PG074, 2022.
- [20] AMD, *Virtex UltraScale+ FPGA Product Tables and Product Selection Guide*, DS923, 2023.